

IFPI · EIXO DE INFORMAÇÃO E COMUNICAÇÃO

20 Desafios **Flex** & Grid

Posicionamento e Layout em HTML/CSS

Flexbox

CSS Grid

CSS3

HTML5

20

DESAFIOS

3

NÍVEIS

10+

CONCEITOS

2h

ESTIMATIVA

Visão geral dos desafios

01 Barra de navegação horizontal

flex · justify-content · gap

FLEX

02 Card com avatar e texto

flex · align-items · gap

FLEX

03 Botões de ação em linha

flex · flex-wrap · gap

FLEX

04 Layout de duas colunas

flex · flex-basis · flex-grow

FLEX

05 Rodapé com colunas

flex · space-between · column

FLEX

06 Centralização perfeita

flex · center · min-height

FLEX

07 Carrinho de compras

flex · align-self · margin-left: auto

FLEX

08 Stepper de etapas

flex · order · flex-direction

FLEX

09 Grade de cards de produto

grid · repeat · auto-fill · minmax

GRID

10 Layout de página completo

grid · template-areas · fr

GRID

11 Galeria fotográfica

grid · span · gap · object-fit

GRID

12 Tabela de preços

grid · column · row · z-index

GRID

13 Dashboard com widgets

grid · dense · auto-rows

GRID

14 Calendário mensal

grid · fixed columns · nth-child

GRID

15 Timeline vertical

grid · lines · pseudo-elements

GRID

16 Mosaic de magazine

grid · named lines · overlap

GRID

17 Layout de blog responsivo

grid externo · flex interno

FLEX + GRID

18 Painel de e-commerce

grid areas · flex cards

FLEX + GRID

19 App de mensagens

grid layout · flex chat bubbles

FLEX + GRID

20 Landing page completa

grid hero · flex nav · grid features

FLEX + GRID

Como usar este material: cada desafio traz uma descrição do que deve ser construído, os requisitos técnicos, os conceitos envolvidos e uma dica visual do layout esperado. Comece pelo HTML semântico, depois aplique o CSS. Use o DevTools do navegador para inspecionar e ajustar em tempo real. Os desafios são progressivos — resolva na ordem para consolidar os conceitos.

Flexbox — O eixo que organiza

Flexbox resolve alinhamento em **uma dimensão** (linha ou coluna). É ideal para barras de navegação, cards com conteúdo interno, grupos de botões e qualquer situação onde os itens precisam se ajustar ao espaço disponível. Pense nele como uma fila inteligente.

#01

Barra de navegação horizontal

Fácil

FLEX

Crie uma barra de navegação com **logo à esquerda**, links no centro e um botão de ação à direita. Os itens devem ser distribuídos horizontalmente com espaçamento uniforme. Em telas menores que 600px, os links devem quebrar para a linha de baixo. Use a propriedade `flex-wrap` para isso.

REQUISITOS HTML

- > Um `<nav>` como container externo
- > Logo dentro de um `<a>`
- > Lista `<ul>` com pelo menos 4 links
- > Botão com texto "Entrar" ou "Cadastrar"

REQUISITOS CSS

- > Container com `display: flex`
- > `justify-content: space-between`
- > `align-items: center`
- > Lista interna também com Flexbox
- > Hover nos links com transição suave

CONCEITOS:

`display: flex`

`justify-content`

`align-items`

`flex-wrap`

`gap`

Card com avatar e texto

#02

Fácil

FLEX

Construa um card de perfil de usuário onde um **avatar circular fica à esquerda** e as informações (nome, cargo, biografia curta) ficam à direita. O avatar deve ter tamanho fixo (64px × 64px) e as informações devem crescer para ocupar o restante do espaço. Adicione uma linha de tags de habilidades na parte inferior do card.

REQUISITOS HTML

- > `<article>` como container do card
- > `<img>` ou `<div>` circular para avatar
- > `<div>` agrupando nome, cargo e bio
- > Linha de badges de habilidades no fim

REQUISITOS CSS

- > Container: `display: flex` horizontal
- > Avatar: `flex-shrink: 0`
- > Info: `flex: 1` para crescer
- > Tags: `flex-wrap: wrap` com gap
- > Avatar com `border-radius: 50%`

CONCEITOS:

`flex: 1`

`flex-shrink: 0`

`align-items`

`flex-wrap`

`gap`

Grupo de botões de ação

#03

Fácil

FLEX

Crie um grupo de botões que representam ações de um editor de texto: Negrito, Itálico, Sublinhado, Separador, Alinhar esquerda, Centro, Direita, Justificar, Separador, Cor da fonte, Cor de fundo. Os botões devem ter **tamanho igual** e quebrar para a linha de baixo quando necessário, sempre mantendo alinhamento à esquerda.

REQUISITOS HTML

- > `<div role="toolbar">` como container
- > Pelo menos 10 `<button>`
- > Separadores visuais entre grupos
- > Ícones ou texto curto em cada botão

REQUISITOS CSS

- > `display: flex; flex-wrap: wrap`
- > `align-items: center`
- > Botões com largura e altura fixas (36px)
- > Estado `:active` visível
- > Separadores: `width: 1px; align-self: stretch`

CONCEITOS:

`flex-wrap`

`align-self`

`align-items: center`

`:active`

#04

Layout de duas colunas (sidebar + conteúdo)

Médio

FLEX

Monte um layout clássico com **sidebar fixa de 260px à esquerda** e área de conteúdo que ocupa todo o restante. A sidebar deve conter um menu de navegação vertical. O conteúdo deve ter pelo menos 3 artigos empilhados. Em mobile (<768px), a sidebar deve ficar acima do conteúdo (mude a direção do flex).

REQUISITOS HTML

- > `<div class="layout">` como wrapper
- > `<aside>` para a sidebar
- > `<main>` para o conteúdo
- > Menu vertical dentro do aside

REQUISITOS CSS

- > Wrapper: `display: flex; gap: 24px`
- > Sidebar: `flex: 0 0 260px` (não cresce, não encolhe)
- > Main: `flex: 1 1 0` (cresce o máximo)
- > Mobile: `flex-direction: column`

CONCEITOS:

flex-grow

flex-shrink

flex-basis

flex-direction

@media

#05

Rodapé com três colunas de links

Fácil

FLEX

Crie um rodapé completo com **três colunas iguais de links** (Empresa, Produto, Suporte), cada uma com um título e lista de links. Abaixo das colunas, uma linha de copyright à esquerda e links de redes sociais à direita. Use `flex-direction: column` dentro de cada coluna para empilhar os links verticalmente.

REQUISITOS HTML

- > `<footer>` como wrapper
- > 3 divs de coluna dentro de um container
- > Cada coluna com `<h4>` e `<ul>`
- > Linha de rodapé separada para copyright

REQUISITOS CSS

- > Container das colunas: `display: flex; gap`
- > Cada coluna: `flex: 1`
- > Lista interna: `flex-direction: column; gap`
- > Rodapé: `justify-content: space-between`

CONCEITOS:

flex-direction: column

flex: 1

space-between

gap

#06

Centralização perfeita (hero section)

Fácil

FLEX

Crie uma seção hero que ocupa **100% da altura da viewport** e centraliza todo o conteúdo (título, subtítulo e botão CTA) tanto horizontalmente quanto verticalmente. O conteúdo interno deve estar em coluna. Este é o padrão mais usado em páginas de apresentação e landing pages.

REQUISITOS HTML

- > `<section class="hero">`
- > `<h1>` com título grande
- > `<p>` com subtítulo
- > Dois `<a>` estilizados como botões

REQUISITOS CSS

- > `display: flex; flex-direction: column`
- > `justify-content: center; align-items: center`
- > `min-height: 100vh`
- > Botões lado a lado com `flex-direction: row`
- > `text-align: center` para textos

CONCEITOS:

`min-height: 100vh``flex-direction: column``justify-content: center``align-items: center`

#07

Item de carrinho de compras

Médio

FLEX

Reproduza uma linha de item de carrinho: imagem do produto à esquerda, nome e descrição no centro, seletor de quantidade, e preço + botão de remover empurrados para a **extrema direita** usando `margin-left: auto`. Crie pelo menos 3 itens dentro de um container scrollável.

REQUISITOS HTML

- > `<ul>` como lista do carrinho
- > Cada `<li>` é um item flex
- > Imagem, texto, input de quantidade, preço, botão
- > Total do carrinho no rodapé da lista

REQUISITOS CSS

- > Cada item: `display: flex; align-items: center`
- > Texto: `flex: 1`
- > Preço: `margin-left: auto`
- > Input quantidade com largura fixa
- > Borda entre itens com `border-bottom`

CONCEITOS:

`margin-left: auto``flex: 1``align-items: center``gap`

#08

Stepper de etapas de processo

Médio

FLEX

Construa um stepper horizontal com 4 etapas (Dados pessoais → Endereço → Pagamento → Confirmação). Cada etapa tem um **círculo numerado**, um rótulo abaixo e uma linha conectando ao próximo. A etapa ativa tem cor destacada, as concluídas ficam com um checkmark, as futuras ficam acinzentadas. Use `order` e pseudo-elementos.

REQUISITOS HTML

- > Container `<ol>` ou `<div>` com 4 filhos
- > Cada passo: círculo + label embaixo
- > Classes para `done`, `active`, `pending` estados:

REQUISITOS CSS

- > Container: `display: flex; justify-content: center`
- > Cada passo: `flex: 1; flex-direction: column; align-items: center`
- > Linha: `::after` no círculo ou `::before` no item
- > Cores por estado via classes CSS

CONCEITOS:

`order``::before / ::after``flex-direction: column``position: relative`

CSS Grid — A grade que estrutura

Grid resolve layout em **duas dimensões** (linhas e colunas) simultaneamente. É a ferramenta certa para estruturar páginas inteiras, galerias, dashboards e qualquer composição onde você precisa controlar posição tanto horizontal quanto verticalmente. Pense nele como uma planta baixa onde você determina exatamente onde cada cômodo fica.

#09

Grade responsiva de cards de produto

Fácil

GRID

Crie uma grade de 8 cards de produto que se **ajusta automaticamente** ao tamanho da tela: 4 por linha em desktop, 2 em tablet, 1 em mobile — sem usar nenhum `@media query`. Use `repeat(auto-fill, minmax(...))` para que o grid faça esse trabalho sozinho.

REQUISITOS HTML

- > Container `<ul>` ou `<div>`
- > 8 itens com imagem, nome, preço e botão
- > Use classes semânticas nos cards

REQUISITOS CSS

- > `display: grid`
- > `grid-template-columns: repeat(auto-fill, minmax(200px, 1fr))`
- > `gap: 16px`
- > Cards com altura igual: `align-items: start` no grid

CONCEITOS:

auto-fill

minmax()

fr

gap

repeat()

#10

Layout de página completo com named areas

Médio

GRID

Monte o esqueleto completo de uma página usando **grid-template-areas**. O layout: header ocupa a largura toda no topo, sidebar à esquerda, main no centro, aside à direita, footer embaixo em toda a largura. Todas as regiões devem ser nomeadas com grid-area. Em mobile, todas as áreas ficam em coluna única.

REQUISITOS HTML

- > `<div class="layout">` wrapper
- > `<header>` , `<nav>` , `<main>` ,
- > `<aside>` , `<footer>`
- > Conteúdo de exemplo em cada área

REQUISITOS CSS

- > `grid-template-areas` com 5 regiões
- > `grid-template-columns: 200px 1fr 200px`
- > `grid-template-rows: auto 1fr auto`
- > Cada filho com `grid-area: nome`

CONCEITOS:

`grid-template-areas``grid-area``grid-template-columns``1fr`

#11

Galeria fotográfica com células de destaque

Médio

GRID

Monte uma galeria de 9 imagens em grid onde **a primeira imagem ocupa 2 colunas e 2 linhas**, criando um efeito de destaque. As demais imagens preenchem o restante normalmente. Use `grid-column: span 2` e `grid-row: span 2`. Todas as imagens devem ter a mesma altura com `object-fit: cover`.

REQUISITOS HTML

- > Container `<figure>` ou `<div>`
- > 9 `<img>` dentro do container
- > Classe especial na primeira imagem
- > Atributo `alt` descritivo em cada imagem

REQUISITOS CSS

- > `grid-template-columns: repeat(3, 1fr)`
- > `grid-auto-rows: 200px`
- > Destaque: `grid-column: span 2; grid-row: span 2`
- > Imagens: `width: 100%; height: 100%; object-fit: cover`

CONCEITOS:

`grid-column: span``grid-row: span``object-fit: cover``grid-auto-rows`

#12

Tabela de preços (pricing table)

Médio

GRID

Construa uma tabela de preços com 3 planos lado a lado (Básico, Pro, Enterprise). O plano do meio (**Pro**) deve ser visualmente maior e se sobrepor levemente aos outros, usando `z-index` e escala. As features devem estar alinhadas horizontalmente entre os planos — use Grid para garantir isso sem tabela HTML.

REQUISITOS HTML

- > Container único para os 3 planos
- > Cada plano: nome, preço, lista de features, botão
- > Classe `featured` no plano do meio
- > Badge "Mais popular" no plano featured

REQUISITOS CSS

- > `grid-template-columns: 1fr 1.15fr 1fr`
- > Plano featured: `transform: scale(1.05)`
- > Plano featured: `z-index: 1`
- > `align-items: start` para manter topo alinhado

CONCEITOS:

`grid-template-columns``z-index``transform: scale()``align-items: start`

#13

Dashboard com widgets de tamanhos variados

Difícil

GRID

Crie um dashboard com 6 widgets de tamanhos diferentes: widget de gráfico grande ocupa 2 colunas, widget de notificações ocupa 2 linhas, demais widgets ocupam 1x1. Use **grid denso** (`grid-auto-flow: dense`) para que o navegador preencha os buracos automaticamente sem ordem manual.

REQUISITOS HTML

- > Container `.dashboard` com 6 widgets
- > Widget grande com classe `.wide`
- > Widget alto com classe `.tall`
- > Conteúdo simulado em cada widget (título + dado)

REQUISITOS CSS

- > `grid-template-columns: repeat(3, 1fr)`
- > `grid-auto-rows: 160px`
- > `grid-auto-flow: dense`
- > `.wide { grid-column: span 2 }`
- > `.tall { grid-row: span 2 }`

CONCEITOS:

`grid-auto-flow: dense``grid-auto-rows``span``grid-column``grid-row`

Calendário mensal

#14

Médio

GRID

Construa o visual de um calendário mensal com 7 colunas (Dom a Sáb) e linhas suficientes para cobrir o mês. O cabeçalho com os dias da semana e as células dos dias devem estar **perfeitamente alinhados** usando o mesmo grid. Use `:nth-child` para colorir os fins de semana.

REQUISITOS HTML

- > Container `<div class="calendar">`
- > Linha de cabeçalho com 7 nomes de dias
- > Células numeradas de 1 a 30/31
- > Células vazias para alinhar o início do mês

REQUISITOS CSS

- > `grid-template-columns: repeat(7, 1fr)`
- > `grid-auto-rows: 80px`
- > Célula de hoje: borda ou fundo destacado
- > Fins de semana: via `:nth-child(7n+1)`

CONCEITOS:

`repeat(7, 1fr)``grid-auto-rows``:nth-child()``grid-column-start`

Timeline vertical de eventos

#15

Difícil

GRID

Crie uma timeline vertical onde eventos alternam entre **lado esquerdo e direito** de uma linha central. Use Grid com 3 colunas: conteúdo-esquerdo | linha central | conteúdo-direito. Os itens pares ficam no `grid-column: 1`, os ímpares no `grid-column: 3`. A linha central é criada com `::before` no container.

REQUISITOS HTML

- > Container `.timeline`
- > 5 a 6 eventos com data, título e descrição
- > Classes `.left` e `.right`
- > Marcador circular no centro de cada evento

REQUISITOS CSS

- > `grid-template-columns: 1fr 40px 1fr`
- > Linha: `::before` no container, posição absoluta
- > Eventos pares: `grid-column: 1`
- > Eventos ímpares: `grid-column: 3`
- > Marcador: `grid-column: 2; justify-self: center`

CONCEITOS:

`grid-column-start``justify-self``position: relative``::before``:nth-child(odd/even)`

#16

Mosaico estilo magazine

Difícil

GRID

Reproduza o layout editorial de uma página de revista: um artigo principal **ocupa 2 colunas e 3 linhas** à esquerda, enquanto 4 artigos menores se encaixam à direita em 2 colunas × 2 linhas, mais uma manchete de rodapé com largura total. Use posicionamento explícito com linhas nomeadas do Grid.

REQUISITOS HTML

- > 6 artigos com imagem, categoria, título e resumo
- > Artigo principal com classe `.hero`
- > Rodapé com classe `.banner`

REQUISITOS CSS

- > `grid-template-columns: repeat(4, 1fr)`
- > `grid-template-rows: repeat(3, 200px) auto`
- > `.hero { grid-column: 1 / 3; grid-row: 1 / 4 }`
- > `.banner { grid-column: 1 / -1 }`

CONCEITOS:

`grid-column: 1 / 3``grid-row: 1 / 4``1 / -1 (span completo)``named lines`

Flex + Grid — O par dinâmico

Na prática, Flex e Grid trabalham **juntos**. A regra é simples: Grid estrutura o esqueleto da página (macro layout), Flexbox alinha o conteúdo dentro de cada componente (micro layout). Os desafios desta seção simulam projetos reais onde você escolhe a ferramenta certa para cada camada.

#17

Layout de blog responsivo

Médio

FLEX + GRID

Monte um blog com header fixo, área de posts em grid e sidebar. **Grid** define o layout externo (posts + sidebar) e o grid de cards. **Flex** organiza o interior de cada card (imagem + metadados + botão empurrado para baixo com `margin-top: auto`).

USE GRID PARA

- › Layout principal: conteúdo + sidebar (2 colunas)
- › Grade de posts: `repeat(2, 1fr)`
- › Posicionamento de sidebar com `grid-row: span`

USE FLEX PARA

- › Header: logo + nav + busca
- › Cada card: `flex-direction: column`
- › Botão "Ler mais" no fundo: `margin-top: auto`
- › Metadados do post: autor + data + categoria

CONCEITOS:

grid externo

flex interno

margin-top: auto

grid-row: span

flex-direction: column

#18

Painel administrativo de e-commerce

Difícil

FLEX + GRID

Construa um admin panel completo: sidebar de navegação fixa à esquerda (Flex vertical), topbar com busca e avatar (Flex horizontal), área de conteúdo com cards de KPI em grid (4 colunas), e tabela de pedidos recentes abaixo. Os cards de KPI têm ícone à esquerda e valor + label à direita (Flex interno).

USE GRID PARA

- > Layout geral: sidebar | área principal
- > `grid-template-areas` para estrutura
- > Cards KPI: `repeat(4, 1fr)`

USE FLEX PARA

- > Sidebar: nav vertical com ícones + labels
- > Topbar: search + notificações + avatar
- > Interior de cada card KPI: ícone + dados
- > Linhas da tabela de pedidos

CONCEITOS:

`grid-template-areas``position: sticky``overflow-y: auto``align-items: center``gap`

#19

Interface de aplicativo de mensagens

Difícil

FLEX + GRID

Recrie a interface de um app de chat tipo WhatsApp Web: lista de conversas à esquerda (sidebar), área de chat à direita com histórico de mensagens e campo de input no rodapé. **Mensagens próprias** ficam à direita (azul), mensagens recebidas à esquerda (cinza). Use `align-self` para isso.

USE GRID PARA

- > Layout: `grid-template-columns: 320px 1fr`
- > Área do chat: `grid-template-rows: auto 1fr auto`
- > (header do chat | mensagens | input)

USE FLEX PARA

- > Lista de conversas: `flex-direction: column`
- > Cada contato: avatar + nome + última msg
- > Container de mensagens: `flex-direction: column; gap`
- > Mensagens: `align-self: flex-end` (próprias)
- > Barra de input: ícone + textarea + botão enviar

CONCEITOS:

`align-self: flex-end``overflow-y: scroll``grid-template-rows``max-width em bubbles``border-radius assimétrico`

#20

Landing page completa de produto SaaS

Difícil

FLEX + GRID

O desafio final: uma landing page profissional com múltiplas seções. **Navbar** com Flex (logo + links + CTA). **Hero** em Grid 2 colunas (texto + imagem). **Features** em Grid 3 colunas com ícone, título e descrição em Flex vertical. **Depoimentos** em Grid auto-fill. **Pricing** do desafio 12 reutilizado. **Footer** do desafio 05 reutilizado.

ESTRUTURA DAS SEÇÕES

- > `nav` : Flex, justify space-between
- > `.hero` : Grid 2 colunas, texto + imagem
- > `.features` : Grid 3 colunas auto-fill
- > `.testimonials` : Grid auto-fill
minmax(280px, 1fr)
- > `.pricing` : Grid 3 colunas (reuse #12)
- > `footer` : Flex colunas (reuse #05)

CRITÉRIOS DE QUALIDADE

- > Responsivo sem quebrar em 320px, 768px e 1280px
- > Sem `position: absolute` para layout
- > Sem tabelas HTML para estrutura visual
- > Sem `float` em lugar nenhum
- > Scroll suave entre seções (`scroll-behavior: smooth`)
- > Código organizado com comentários por seção

CONCEITOS:

composição de layouts

@media queries

scroll-behavior

CSS custom properties

reutilização

Propriedades essenciais

● Flexbox — Container

<code>display</code>	<code>flex</code> <code>inline-flex</code>
<code>flex-direction</code>	<code>row</code> <code>column</code> <code>row-reverse</code> <code>column-reverse</code>
<code>justify-content</code>	<code>flex-start</code> <code>center</code> <code>flex-end</code> <code>space-between</code> <code>space-around</code> <code>space-evenly</code>
<code>align-items</code>	<code>stretch</code> <code>flex-start</code> <code>center</code> <code>flex-end</code> <code>baseline</code>
<code>flex-wrap</code>	<code>nowrap</code> <code>wrap</code> <code>wrap-reverse</code>
<code>gap</code>	<code>row-gap</code> <code>column-gap</code> (ou shorthand)

● Flexbox — Filho (item)

<code>flex</code>	<code>grow shrink basis</code> (shorthand)
<code>flex-grow</code>	0 (não cresce) 1 (cresce)
<code>flex-shrink</code>	1 (encolhe) 0 (não encolhe)
<code>flex-basis</code>	<code>auto</code> <code>px</code> <code>%</code> 0
<code>align-self</code>	<code>auto</code> <code>flex-start</code> <code>center</code> <code>flex-end</code> <code>stretch</code>
<code>order</code>	0 (padrão) número inteiro

● CSS Grid — Container

<code>display</code>	<code>grid</code> <code>inline-grid</code>
<code>grid-template-columns</code>	<code>repeat(3, 1fr)</code> <code>200px auto 1fr</code>
<code>grid-template-rows</code>	<code>auto</code> <code>100px</code> <code>minmax(100px, auto)</code>
<code>grid-template-areas</code>	<code>"header header" "sidebar main" "footer footer"</code>
<code>grid-auto-flow</code>	<code>row</code> <code>column</code> <code>row dense</code> <code>column dense</code>
<code>gap</code>	<code>row-gap</code> <code>column-gap</code> (ou shorthand)

● CSS Grid — Filho (item)

<code>grid-column</code>	1 / 3 <code>span 2</code> 1 / -1
<code>grid-row</code>	1 / 3 <code>span 2</code> <code>auto</code>
<code>grid-area</code>	nome-da-area <code>row-start</code> / <code>col-start</code> / <code>row-end</code> / <code>col-end</code>
<code>justify-self</code>	<code>start</code> <code>center</code> <code>end</code> <code>stretch</code>
<code>align-self</code>	<code>start</code> <code>center</code> <code>end</code> <code>stretch</code>
<code>place-self</code>	<code>align-self</code> / <code>justify-self</code> (shorthand)

Regra de ouro: Escolha **Grid** quando você pensa em *linhas E colunas ao mesmo tempo* (layout de página, galeria, dashboard). Escolha **Flex** quando você pensa em *uma linha ou uma coluna* (navbar, card interno, grupo de botões). Na dúvida, `display: grid` resolve quase tudo — mas Flex é mais simples para alinhamentos pontuais.